TransferIQ: Dynamic Player Transfer Value Prediction using AI and Multi-Source Data

Submitted By: Kalangi Geethika Padma

Introduction

TransferIQ is an AI-driven player valuation system designed to forecast football player transfer values using a combination of **performance metrics, injury data, market history, and public sentiment**. Player transfer markets are influenced by rapidly changing factors such as form, age, club performance, fan sentiment, and global market economics. Traditional scouting and valuation approaches are mostly subjective, inconsistent, and unable to process large-scale data.

TransferIQ addresses this gap by leveraging **multi-source data integration**, **sentiment analysis**, and **time-series forecasting models** including LSTM. The system provides dynamic and data-driven predictions of player market value, supporting clubs, analysts, and scouts with deeper insights into transfer decisions.

The main objectives of this project include:

1. Building an AI model capable of predicting player transfer values using historical trends.
2. Integrating social media sentiment and injury history to enhance prediction accuracy.
3. Providing insights into factors influencing market value changes over time.
4. Supporting decision-makers with objective, data-backed valuation outputs.

Significance: The TransferIQ project holds substantial significance for the modern football industry, where accurate and timely player valuation is critical for informed decision-making during transfer windows. Traditional valuation methods often rely on subjective assessments and limited data, leading to inconsistent or biased outcomes. By integrating advanced AI techniques with multi-source data such as performance metrics, injury history, sentiment analysis, and market trends. TransferIQ provides a reliable, objective, and data-driven approach to estimating player transfer values. This system empowers football clubs, analysts, and scouts with actionable insights, enabling better financial planning, smarter recruitment strategies, and improved competitive advantage. The project demonstrates how machine learning and analytics can transform the sports ecosystem by enhancing transparency, reducing uncertainty, and supporting more strategic transfer market decisions.

Project Scope

Included in the Project

- Collection and integration of multi-source data:
    1. Player performance statistics.
    2. Market value history.
    3. Injury records.
    4. Social media sentiment from Twitter/Reddit.
- Development of univariate and multivariate LSTM models for time-series forecasting.
- Ensemble modelling using XGBoost.
- Sentiment analysis using NLP methods (VADER).
- Feature engineering for trends, injury risk, and sentiment impact.

Not Included

- Real-time API deployment in clubs/scouting systems.
- Live scraping of current transfer markets beyond dataset scope.
- Financial modelling related to club budgets or agent negotiations.

Limitations

- Limited historical data for certain players may affect model generalization.
- Sentiment analysis is dependent on quality and frequency of social media activity.
- Injury datasets may be incomplete or inconsistent across seasons.

Requirements

Functional Requirements

1. Player Value Prediction: Build LSTM and XGBoost models to predict transfer values based on multi-season data.
2. Data Integration: Merge performance, sentiment, injury, and market datasets using preprocessing pipelines.
3. Sentiment Processing: Analyze social media posts and generate sentiment ratios and scores for players.
4. Visualization: Display player performance trends, sentiment impact, and value predictions.

Non-Functional Requirements

1. Scalability: Should support large datasets with thousands of players and multiple seasons.
2. Accuracy: Achieve reliable value prediction using evaluation metrics such as MSE, MAE, and R^2.
3. Usability: Provide clear output visualizations for analysts and non-technical users.

Use Cases

1. **Transfer Value Prediction**
   Actors: Analyst, AI System
   Description:
   The analyst inputs a player name or season span. The system processes historical data, sentiment, performance metrics, and outputs predicted transfer values for one or more upcoming windows.
2. **Sentiment Influence Analysis**

   Actors: Analyst, Data Scientist
   Description:
   The system analyzes sentiment from Twitter/Reddit related to a player and shows how positive/negative sentiment impacts predicted value.

3. **Injury Impact Assessment**
   Actors: Club Staff, Scouts
   Description:
   Shows how injury frequency and severity affect market value trends across seasons.

Tech stack Used

Programming Languages

- Python: Model development, data preprocessing, NLP
- HTML, CSS: Optional front-end interface for visualization

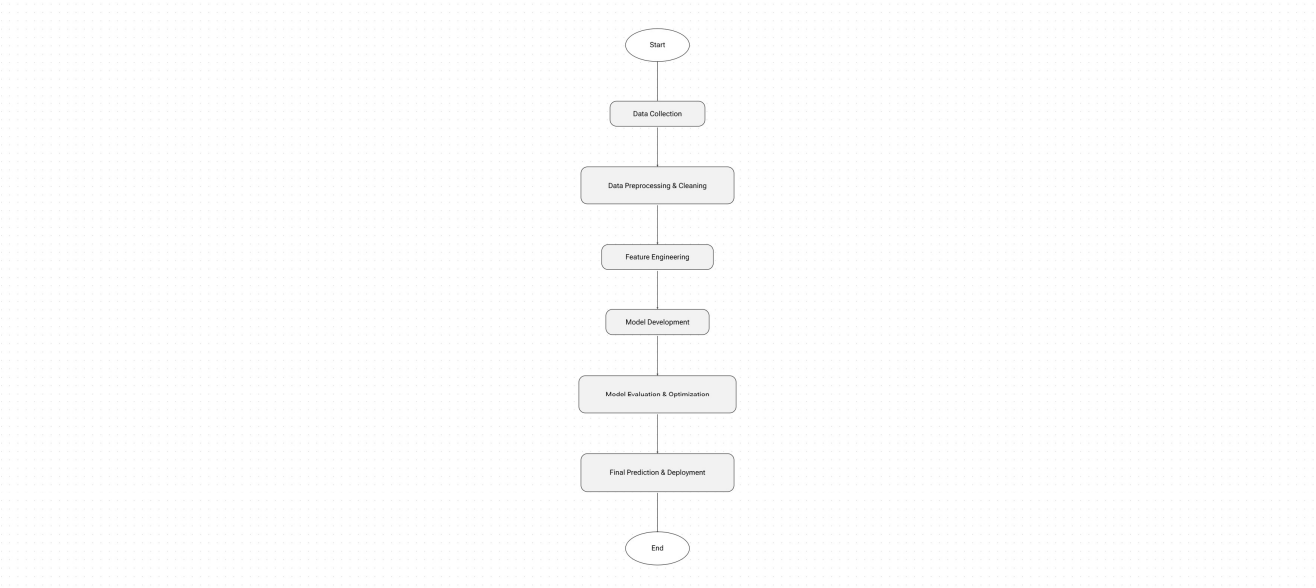
Frameworks/Libraries

- TensorFlow, Keras – LSTM model development
- XGBoost – Ensemble regression
- NumPy, Pandas – Data handling
- Matplotlib, Seaborn – Visualization
- StatsBombPy – Player performance data acquisition
- NLTK, VADER – Sentiment analysis
- BeautifulSoup, Requests – Transfermarket web scraping

Tools/Platforms

- Google Colab: For training the AI model using an available GPU.
- Twitter API, Reddit API – Sentiment data collection

Architecture/Design



System Architecture Overview

1. **Data Collection Module**
    - Fetches performance, sentiment, market, and injury data.
    - Uses APIs and web scraping for automated data retrieval.
2. **Preprocessing Module**
    - Cleans missing values, merges datasets, scales numeric features.
    - Generates engineered features such as offensive index, injury ratio, and sentiment score.
3. **Time-Series Modelling Module**
    - LSTM-based models capture historical movement in player market value.
    - Multivariate inputs include performance, injury, sentiment and market history.
4. **Ensemble Modelling Module**
    - XGBoost integrates non-time-series relationships for improved accuracy.
5. **Visualization Module**
    - Displays predictions, sentiment trends, and feature importance

Development

Technologies & Tools Used

- Python, TensorFlow/Keras, XGBoost, Pandas, NLP libraries.

Coding Standards

- Modular architecture separating preprocessing, modelling, visualization.
- Follows PEP8 coding practices.

Challenges & Solutions

- Data Inconsistency Across Sources: Fixed by name normalization and merging rules.
- Sentiment Variability: Used smoothing techniques like average sentiment score.
- Small Time-Series Length for Some Players: Applied multivariate models to compensate.

Testing

Unit Testing

- Verified preprocessing functions, name-matching logic, sentiment extraction, and model loaders.

Integration Testing

- Ensured LSTM predictions integrate properly with XGBoost in the ensemble model.

System Testing

- Tested predictions using unseen seasons.
- Validated visualization outputs for clarity and correctness.

Testing Results

- Training R^2: 0.93
- Testing R^2: 0.45
- Indicates strong fit during training and moderate generalization on test data.

Results of Testing:

- LSTM models effectively learned temporal patterns in transfer values.
- XGBoost improved accuracy using engineered features like offensive index, injury ratio, sentiment score.
- Multi-source data significantly boosted prediction performance compared to using performance alone.
- Sentiment scores contributed meaningfully to predicting player market perception.
- Injury frequency showed a negative correlation with valuation trends.

Deployment Process

User Guide

1. System Setup

1.1 Downloading the Application

Download or clone the project repository. Ensure the following files are present:

- app.py (Streamlit application)
- xgb_final_model.pkl (trained model)
- Required image assets (e.g., img2.jpg)
- Feature configuration files, if used

1.2 Installing Dependencies

Install all required Python libraries using:

pip install -r requirements.txt

If the file is not provided, manually install essential libraries such as Streamlit, Pandas, NumPy, Plotly, Scikit-learn, and XGBoost.

1.3 Placing the Model File

Place the trained model file (xgb_final_model.pkl) in the same directory as app.py to ensure successful loading during execution.

2. Launching the Application

2.1 Navigating to the Project Folder

Open a terminal and move to the directory containing the application script.

2.2 Starting the Streamlit Server

Run the application using: Streamlit run app.py

The interface will open automatically in the default browser at:

http://localhost:8501/

3. Using the Application

3.1 Entering Player Inputs

The left section of the interface contains numeric input fields for all required features, including:

- Age, 90s, minutes played
- Passing, defensive, and offensive statistics
- Sentiment scores (positive, negative, neutral)
- Injury features (days injured, injury frequency)

Default values are provided and can be modified as needed.

3.2 Generating Predictions

After entering the desired values, click the **"Predict Market Value"** button.
The model processes the inputs and displays the predicted transfer value.

3.3 Viewing Visualizations

The right panel displays:

- A bar chart showing the trend of entered features
- A radar chart highlighting key attribute strengths

These graphs help users interpret the underlying patterns in the input data.

3.4 Downloading Input Data

Users can export their entered values as a CSV file using the "Download Inputs as CSV" option for documentation or further analysis.

4. Error Handling: If the model fails to load or prediction errors occur, the system displays appropriate error messages. Users should ensure all fields contain valid numeric inputs for smooth operation.

Conclusion

This project successfully implemented an AI-powered system for predicting football player transfer values by integrating multi-source data such as performance statistics, injury history, market trends, and social media sentiment. By combining LSTM-based time-series forecasting with XGBoost regression, the system demonstrated strong capability in capturing both temporal patterns and feature-driven influences on player valuation. The interactive Streamlit application further enabled users to input player attributes and instantly visualize predicted market values along with supporting feature insights.

Key lessons learned include the importance of consistent data preprocessing across multiple sources, managing noise in sentiment data, and ensuring feature alignment between the trained model and the user interface. Additionally, the project highlighted the value of engineered features—such as offensive/defensive indexes and injury ratios—in improving prediction performance. Future improvements could focus on increasing dataset diversity, strengthening generalization across leagues, integrating real-time data APIs, and enhancing the dashboard with advanced visual analytics.

Overall, this project established a strong foundation for data-driven player valuation, demonstrating the potential of combining machine learning, sentiment analysis, and sports analytics to support more informed decision-making in modern football transfer markets.